LEZIONE

Strumenti di diagnostica per sistemi Linux

Docente: Wolfgang Cecchin

1. L'approccio a strati

Quando un sistema non funziona, la tentazione è provare soluzioni a caso. L'approccio professionale è diverso: si ragiona dal basso verso l'alto, verificando un livello alla volta.

```
Livello 4 — Applicazione     WordPress, configurazione virtual host
Livello 3 — Servizio         nginx, MariaDB, PHP-FPM, SSH
Livello 2 — Firewall         iptables, policy, regole
Livello 1 — Rete             interfacce, IP, routing, DNS
```

> *Non ha senso indagare il livello superiore se quello inferiore è rotto. Se la rete non funziona, il firewall è irrilevante. Se il firewall blocca tutto, il servizio non è raggiungibile.*

2. Livello 1 — Diagnostica di rete

Interfacce e indirizzi IP

```
ip a                          # tutte le interfacce con IP
ip a show enp0s8              # una interfaccia specifica
ip link show                  # stato UP/DOWN delle interfacce
```

Cosa cercare: l'interfaccia è UP? Ha un indirizzo IP? L'indirizzo è quello giusto? La netmask è corretta?

Routing

```
ip route show                 # tabella di routing
ip r                          # forma abbreviata
```

Cosa cercare: c'è un default gateway? Punta all'indirizzo giusto?

Connettività

```
ping -c 4 10.0.0.2            # raggiungibilità di un host sulla LAN
ping -c 4 8.8.8.8             # raggiungibilità di internet
traceroute 8.8.8.8            # percorso dei pacchetti
```

Cosa cercare: il ping risponde? Se no, il problema è locale (mittente) o remoto (destinatario)? Traceroute mostra dove si ferma il traffico?

Configurazione persistente

```
cat /etc/network/interfaces   # configurazione Debian
```

Cosa cercare: le righe auto, iface, address, netmask, gateway corrispondono a quello che ci aspettiamo?

DNS

```
cat /etc/resolv.conf          # server DNS configurati
cat /etc/hosts                # risoluzione locale
```

3. Livello 2 — Diagnostica del firewall

Visualizzare le regole attive

```
sudo iptables -L -n --line-numbers     # regole leggibili con numeri di riga
sudo iptables -L INPUT -n -v           # solo catena INPUT, con contatori
sudo iptables-save                     # dump completo in formato restore
```

L'opzione `-n` è importante: senza di essa, iptables tenta di risolvere ogni IP in un nome DNS, rallentando enormemente l'output.

Cosa cercare

Policy di default: la prima riga dice Chain INPUT (policy DROP) o (policy ACCEPT)? Con policy DROP, tutto ciò che non è esplicitamente permesso viene scartato.

Ordine delle regole: iptables le legge dall'alto verso il basso e si ferma alla prima che corrisponde. Una regola di DROP prima di una di ACCEPT vince.

Porte aperte: ci sono regole ACCEPT per le porte dei servizi necessari (22/2222 per SSH, 80 per HTTP, 3306 per MariaDB)?

Traffico di ritorno: c'è una regola per ESTABLISHED,RELATED? Senza di essa, le risposte alle connessioni in uscita vengono bloccate (es. apt update non funziona).

File di persistenza

```
cat /etc/iptables/rules.v4    # regole salvate (iptables-persistent)
```

> *Se le regole in memoria (iptables -L) sono diverse da quelle nel file, al prossimo riavvio si torna alla versione del file.*

4. Livello 3 — Diagnostica dei servizi

Stato dei servizi

```
sudo systemctl status nginx             # stato di nginx
sudo systemctl status mariadb           # stato di MariaDB
sudo systemctl status ssh               # stato di SSH
sudo systemctl status php8.2-fpm        # stato di PHP-FPM
```

Cosa cercare: il servizio è active (running) oppure failed? Se failed, il messaggio d'errore dice cosa è andato storto.

Porte in ascolto

```
sudo ss -tulnp                          # tutte le porte in ascolto
sudo ss -tulnp | grep :80              # solo la porta 80
sudo ss -tulnp | grep :3306            # solo la porta 3306
```

Questo comando è fondamentale. Mostra non solo se un servizio è in ascolto, ma anche su quale indirizzo:

Output	Significato
`0.0.0.0:80`	nginx ascolta su tutte le interfacce (OK)

`127.0.0.1:3306`	MariaDB ascolta solo sulla loopback (non raggiungibile dalla rete)
`10.0.0.2:3306`	MariaDB ascolta solo sulla rete interna (corretto per il backend)

Test di configurazione

```
sudo nginx -t                           # verifica sintassi nginx
sudo sshd -t                            # verifica sintassi SSH
sudo nginx -T                           # mostra configurazione completa nginx
```

Questi comandi controllano la sintassi senza toccare il servizio in esecuzione. Sono sicuri da eseguire in qualsiasi momento.

Log

```
sudo journalctl -u nginx --no-pager -n 30     # ultimi 30 log di nginx
sudo journalctl -u mariadb --no-pager -n 30   # ultimi 30 log di MariaDB
sudo journalctl -u ssh --no-pager -n 30       # ultimi 30 log di SSH
```

I log spesso contengono il messaggio d'errore esatto che spiega perché un servizio non parte o non risponde.

5. Livello 4 — Diagnostica applicativa

Test di connessione al database

```
mysql -h 10.0.0.2 -u wpuser -p          # connessione remota a MariaDB
```

Se fallisce, la causa può essere a qualsiasi livello inferiore: rete (IP sbagliato), firewall (porta bloccata), servizio (bind-address sbagliato) o permessi MySQL (utente non autorizzato per quell'IP).

Test del web server

```
curl -I http://localhost                 # header di risposta di nginx
curl -s http://localhost | head -20      # primi 20 righe della pagina
```

Se nginx risponde localmente ma non dall'esterno, il problema è al livello firewall.

Configurazione nginx

```
sudo nginx -T | grep root                # percorso della document root
sudo nginx -T | grep server_name         # nome a cui risponde nginx
sudo nginx -T | grep php                 # configurazione PHP-FPM
ls -la /var/www/wordpress/               # contenuto della directory WordPress
```

Nome del sito (server_name)

Se il `server_name` non corrisponde a quello che digitiamo nel browser, nginx può servire il sito sbagliato. Il nome deve coincidere con quello impostato nel file hosts del client.

URL di WordPress nel database

WordPress salva il proprio indirizzo nella tabella wp_options. Se nginx risponde ma WordPress fa redirect a un URL diverso, il problema è nel database:

```
mysql -h 10.0.0.2 -u wpuser -p wordpress

SELECT option_name, option_value FROM wp_options
WHERE option_name IN ('siteurl', 'home');
```

I campi `siteurl` e `home` devono contenere l'URL con cui si vuole raggiungere il sito. Se contengono un nome diverso, WordPress genera redirect e link che puntano all'indirizzo sbagliato.

6. Buone pratiche di troubleshooting

Cambiare una cosa alla volta. Dopo ogni modifica, verificare se il problema è risolto prima di passare alla successiva. Cambiare tre cose insieme e poi scoprire che funziona non insegna quale fosse il problema.

Documentare prima, agire poi. Prima di modificare un file, annotare lo stato attuale. Un cat o un iptables -L prima dell'intervento permette di tornare indietro se la modifica peggiora la situazione.

Leggere gli errori. Connection refused è diverso da Connection timed out: il primo indica che il servizio non è in ascolto, il secondo che un firewall sta bloccando il traffico.

Il comando tee

Nella gestione di iptables incontrerete spesso questa sintassi:

```
sudo iptables-save | sudo tee /etc/iptables/rules.v4
```

`tee` legge dallo standard input e scrive contemporaneamente su un file e sullo standard output — come un raccordo a T idraulico. Il motivo per cui si usa `sudo tee` invece del redirect `>` è che il redirect viene eseguito dalla shell prima di sudo, quindi non ha i permessi di root per scrivere sul file. Con tee che gira come root tramite sudo, il problema non si pone.

7. Riepilogo comandi

Obiettivo	Comando
Interfacce e IP	`ip a`
Routing	`ip route show`
Connettività	`ping, traceroute`
Config persistente	`cat /etc/network/interfaces`
Regole firewall	`sudo iptables -L -n --line-numbers`
Stato servizi	`sudo systemctl status <servizio>`
Porte in ascolto	`sudo ss -tulnp`
Test sintassi nginx	`sudo nginx -t`
Log servizi	`sudo journalctl -u <servizio> -n 30`
Test DB remoto	`mysql -h IP -u utente -p`
Test web locale	`curl -I http://localhost`
Config nginx completa	`sudo nginx -T`
URL WordPress	`SELECT ... FROM wp_options`
Salvare regole iptables	`sudo iptables-save | sudo tee ...`